

memory $\longleftrightarrow$ compute: Horace He's blog post:
read/write

//
gelu implementation: $\left\{ \begin{array}{l} \text{NVTX annotations: annotated areas called range} \\ \text{with torch.autograd.profiler.emit_nvtx}; \\ \text{with nvtx.annotate "Forward pass"}; \\ \text{output} = \text{model(input)} \end{array} \right.$

#1: default pytorch implementation (fused) $= y_1 = \text{pytorch-gelu}(x)$

#2: write by our own (not fused) $= y_2 = \text{manual-gelu}(x)$

$\Rightarrow \text{check_equal}(y_1, y_2) = \text{yes}$

$\Rightarrow$ do benchmarking: check running time

#1: $\text{pytorch-time} = 1.13 \text{ ms}$

#2: $\text{manual-time} = 8.15 \text{ ms}$

$\Rightarrow$ let's do profiling: check under the hood $\Rightarrow$ where is time spent?

#1: $\text{pytorch-gelu-profile} = \text{profile}(\text{"pytorch-gelu"}, \text{run_operation}(\text{dim}=16384, \text{operation}=\text{pytorch-gelu}))$

#2: $\text{manual-gelu-profile} = \text{profile}(\text{"manual-gelu"}, \text{run_operation}(\text{dim}=16384, \text{operation}=\text{manual-gelu}))$

#3: write by our own using cuda implementation

$\hookrightarrow$ an extension of C/C++ API.
CUDA implementation is good for: manual, softmax, RMSnorm.

$\text{cuda-gelu-src} = \text{open}(\text{"gelu.cu"}). \text{read}()$

include <math.h> # include <torch/extension.h> # include ---